

MEDIATEK

MT8668 Yocto Camera Driver User Manual

版本: 1.0

出版日期: 2026-1-28

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2026-1-28	江攀	正式版

目录

版本记录 2

目录 3

1 Camera Driver 4

 1.1 概述 4

 1.2 架构/进程概述 4

 1.3 配置/客制化指南 4

 1.3.1 如何添加一个新的 Sensor 5

附件一 附加条款 24

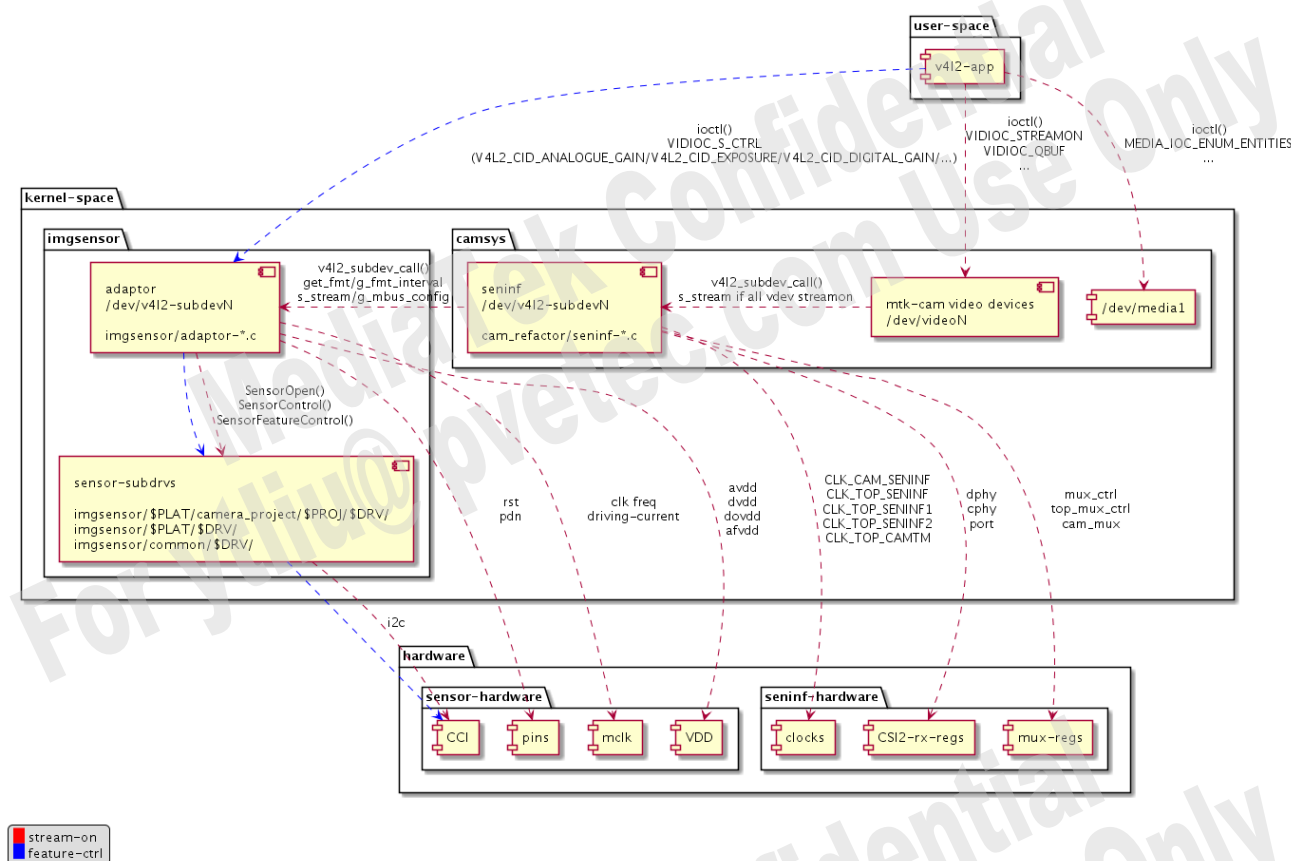
1 Camera Driver

1.1 概述

本章节主要说明 MT8668 Camera 移植方法以及相关问题的调试方法和思路。

1.2 架构/进程概述

Camera driver 主要分 User space 和 Kernel space 两部分，两边通过 IOCTL 传递参数。Raw sensor metadata 等放在 User space；sensor 的相关设定和 power control 是在 Kernel space。



1.3 配置/客制化指南

本节主要介绍 Camera driver 移植指南。

1.3.1 如何添加一个新的 Sensor

这部分对于 YUV 和 Raw sensor 步骤基本一致。

1.3.1.1 移植文件列表

- **Config Files**
 - \src\kernel\linux\v6.12_mt8668\co_device_module\kernel\configs\auto_yct.config
- **Kernel Space**
 - \src\kernel\linux\v6.12_mt8668\co_device_module\drivers\misc\mediatek\imgsensor\inc\kd_imgsensor.h
 - \src\kernel\linux\v6.12_mt8668\co_device_module\arch\arm64\boot\dts\mediatek\cust_mt8668_camera_v4l2.dtsi
 - \src\kernel\modules\mt8668\camera\imgsensor\src_spm-isp8s\
- **User Space**
 - \src\multimedia\mtkcam-mt8668\mtkcam-utils\kernel-headers\mediatek\kd_imgsensor.h
 - \src\multimedia\camera-hal\mt8668\custom\common\hal\imgsensor_metadata\sensor\
 - \src\multimedia\camera-hal\mt8668\custom\mt6881\hal\imgsensor_metadata\

1.3.1.2 修改 Config 文件

- \src\kernel\linux\v6.12_mt8668\co_device_module\kernel\configs\auto_yct.config
 - CONFIG_CUSTOM_KERNEL_IMGSENSOR="xxxx_mipi_raw xxxx_mipi_raw xxxx_mipi_yuv"

在上面增加新的 sensor name。

1.3.1.3 在 Kernel 中添加 Sensor 文件

- \src\kernel\modules\mt8668\camera\imgsensor\src_spm-isp8s\common\\${CamDrv}\

参考其他 sensor，在上面目录增加自己的 sensor driver 的目录，命名格式可以参考下图：

src > kernel > modules > mt8668 > camera > imgsensor > src_spm-isp8s > common > max96712_mipi_yuv

名称	修改日期	类型	大小
Makefile	2025/11/17 19:22	檔案	1 KB
max96712mipiyuv_Sensor	2025/11/17 19:22	C Source	27 KB
max96712mipiyuv_Sensor	2025/11/17 19:22	C/C++ Header	6 KB

1.3.1.4 添加 Sensor ID 和 Sensor Name

- \src\multimedia\mtkcam-auto\mtkcam-utils\kernel-headers\mediatek\mt8668\kd_imgsensor.h
- \src\kernel\linux\v6.12_mt8668\co_device_module\drivers\misc\mediatek\imgsensor\inc\kd_imgsensor.h

在以上两个文件内都添加 sensor ID 和 sensor name:

- 添加 sensor ID

```
392 #define MAX96712MIPI_SENSOR_ID 0x52
```

- 添加 sensor name

```
642 #define SENSOR_DRVNAME_MAX96712_MIPI_YUV "max96712_mipi_yuv"
```

1.3.1.5 修改 dts 文件

\src\kernel\linux\v6.12_mt8668\co_device_module\arch\arm64\boot\dts\mediatek\cust_mt8668_camera_v4l2.dtsi

通过分析硬件原理图找到当前 sensor 是挂在哪个 I2C 下面, 然后在对应的 I2C 下面增加 sensor 配置, 如下所示:

- 增加了 sensor0



关于 seninf 和 sensor 的关联, 可以参考下图, seninf_top 里 csi-port 的值是硬件实际连接的 csi-port。

clocks = <&cksys_clk CLK_CK_UNIVPLL_192M_D32>,
 <&cksys_clk CLK_CK_UNIVPLL_192M_D16>,
 <&cksys_clk CLK_CK_F26M_CK_D2>,
 <&cksys_clk CLK_CK_UNIVPLL_192M_D10>,
 <&cksys_clk CLK_CK_UNIVPLL_192M_D8>,
 <&cksys_clk CLK_CK_UNIVPLL_D6_D16>,
 ① <&cksys_clk CLK_CK_UNIVPLL_D6_D8>,
 <&vlp_cksys_clk CLK_VLP_CK_CAMTG1_SEL>;
 clock-names = "6", "12", "13", "19.2", "24", "26", "52", "mclk";
 status = "okay";

port {
 sensor1_out: endpoint {
 remote-endpoint = <&seninf_csi_port_0_in>;
 };
};

②

Set endpoint

vlp_cksys_clk CLK_VLP_CK_CAMTG0_SEL	mclk0
vlp_cksys_clk CLK_VLP_CK_CAMTG1_SEL	mclk1
vlp_cksys_clk CLK_VLP_CK_CAMTG2_SEL	mclk2
vlp_cksys_clk CLK_VLP_CK_CAMTG3_SEL	mclk3
.....	

```

&seninf_top {
    seninf_csi_port_0: seninf_csi_port_0 {
        compatible = "mediatek,seninf";
        csi-port = "0";
        nvmem-cells = <&csi_efuse0>;
        nvmem-cell-names = "rg_csi";
        port {
            ③ seninf_csi_port_0_in: endpoint {
                remote-endpoint = <&sensor1_out>;
            };
        };
    };
};
  
```

- CLK_VLP_CK_CAMTG1_SEL is mclk source, currently set to mclk1.
- The setting of csi port represents which CSI the current sensor is connected to. For example, 0, 1, 2, 3, 0A, 0B, 1A, 1B etc.
- The port setting Check seninf's csi-port property, currently sensor1 connect to CSI0.

与 dts 里配置对应的就是上电时序的部分（如下所示），需要参考 sensor 规格书中的上电时序来配置。

• Check power on sequence

```

common/imx766_mipi_raw/imx766mipiraw_Sensor.c
static struct subdrv_pw_seq_entry pw_seq[] = {
    {HW_ID_MCLK, {24, 0}, Frequency},
    {HW_ID_RST, {0, 0}},
    {HW_ID_AVDD, {2800000, 0}},
    {HW_ID_AFDVDD, {2800000, 0}},
    {HW_ID_DVDD, {1100000, 0}},
    {HW_ID_DOVDD, {1800000, 1800000}, 1000},
    {HW_ID_MCLK_DRIVING_CURRENT, {, 1000}},
    {HW_ID_RST, {, 3000}},
};
const struct subdrv_entry imx766_mipi_raw_entry = {
    .name = "imx766_mipi_raw",
    .id = IMX766_SENSOR_ID,
    .pw_seq = pw_seq,
    .pw_seq_cnt = ARRAY_SIZE(pw_seq),
    .ops = &ops,
};
  
```

PowerType

Frequency

Current

Voltage min/max

Delay

order

1. Add power sequence info

Important change

- { power_type, {voltage min,voltage max}, delay_time in us }
- Voltage max is equal to min if not specifying or less than min
- Power up sequence from top to bottom
- Power off sequence from bottom to top
- Set mclk frequency (24M) and driving current (4mA)

2. Add subdrv entry

- Naming rule: \$(folder-name)_entry
- Macro will generate sensor-list automatically

```
#define SENSOR_DRVNAME IMX766 MIPI RAW "imx766_mipi_raw"
```

- Rename a copy according to the existing sensor driver source code, and the part of the register and sensor info should be confirmed with the vendor.

以 max96712 为例:

```

static struct subdrv_pw_seq_entry pw_seq[] = {
    {HW_ID_PDN, {1800000, 1800000}, 20000},
};
const struct subdrv_entry max96712_mipi_yuv_entry = {
    .name = "max96712_mipi_yuv",
    .id = MAX96712_MIPI_SENSOR_ID,
    .pw_seq = pw_seq,
    .pw_seq_cnt = ARRAY_SIZE(pw_seq),
    .ops = &ops,
};
  
```

上电时序

sensor name和id

注：需要在 sensor node 下新增一个属性：**mediatek,legacy-search**;

```
sensor2: sensor2@52 {
    mediatek,legacy-search;
    compatible = "mediatek,imgsensor";
    sensor-names = "max96712_mipi_yuv";
    reg = <0x52>;

    #thermal-sensor-cells = <0>;
    dvdd-supply = <&gpio165_regulator>;
    pdn-supply = <&gpio163_regulator>;
    avdd1-supply = <&ioe20_regulator>;
    avdd2-supply = <&ioe21_regulator>;
    avdd3-supply = <&ioe22_regulator>;
    avdd4-supply = <&ioe23_regulator>;

    clocks = <&cksys_reg_clk CLK_CKSYS_REG_UNIVPLL_192M_D32>,
            <&cksys_reg_clk CLK_CKSYS_REG_UNIVPLL_192M_D16>,
            <&cksys_reg_clk CLK_CKSYS_REG_F26M_CK_D2>;
}
```

1.3.1.6 增加新的 Meta 文件

参考 max96712_mipi_yuv 文件进行修改添加，可以直接复制，然后修改文件名和文件内的 sensor name。

- \src\multimedia\camera-hal\mt8668\custom\common\hal\imgsensor_metadata\sensor\
- \src\multimedia\camera-hal\mt8668\custom\mt6881\hal\imgsensor_metadata\

如需增加新的分辨率，可以参考以下代码。

```
C config_static_metadata_project.h X
src > multimedia > camera-hal > mt8668 > custom > mt6881 > hal > imgsensor_metadata > max96712_mipi_yuv > C config_static_metadata_project
70 #if 0 // no AOSP 120fps for 8195
87 CONFIG_ENTRY_VALUE( 120, MINT32) // height
88 CONFIG_ENTRY_VALUE( 120, MINT32) // fps_min
89 CONFIG_ENTRY_VALUE( 120, MINT32) // fps_max
90 CONFIG_ENTRY_VALUE( 4, MINT32) // batch_size_max
91 #endif //
92 CONFIG_METADATA_END()
93
94 CONFIG_METADATA_BEGIN(MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_WITH_DURATIONS)
95 CONFIG_ENTRY_VALUE(NSCam::eImgFmt_BLOB, MINT64)
96 CONFIG_ENTRY_VALUE(1280, MINT64) // width
97 CONFIG_ENTRY_VALUE( 720, MINT64) // height
98 CONFIG_ENTRY_VALUE(MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT, MINT64) // output
99 CONFIG_ENTRY_VALUE(33333333, MINT64) // frame duration
100 CONFIG_ENTRY_VALUE(0, MINT64) // stall duration
101
102 CONFIG_ENTRY_VALUE(NSCam::eImgFmt_RAW16, MINT64)
103 CONFIG_ENTRY_VALUE(1280, MINT64) // width
104 CONFIG_ENTRY_VALUE( 720, MINT64) // height
105 CONFIG_ENTRY_VALUE(MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT, MINT64) // output
106 CONFIG_ENTRY_VALUE(33333333, MINT64) // frame duration
107 CONFIG_ENTRY_VALUE(0, MINT64) // stall duration
108
109 CONFIG_ENTRY_VALUE(NSCam::eImgFmt_YCBCR_420_888, MINT64)
110 CONFIG_ENTRY_VALUE(1280, MINT64) // width
111 CONFIG_ENTRY_VALUE( 720, MINT64) // height
112 CONFIG_ENTRY_VALUE(MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT, MINT64) // output
113 CONFIG_ENTRY_VALUE(33333333, MINT64) // frame duration
114 CONFIG_ENTRY_VALUE(0, MINT64) // stall duration
115
116 CONFIG_ENTRY_VALUE(NSCam::eImgFmt_YCBCR_420_888, MINT64)
117 CONFIG_ENTRY_VALUE( 960, MINT64) // width
118 CONFIG_ENTRY_VALUE( 540, MINT64) // height
```

1.3.1.7 增加新的 Meta 文件(json)

/src/multimedia/camera-hal/mt8668/mtkcam-core/external/firmware/metadata 下参考 max96712_mipi_yuv 文件进行修改添加，可以直接复制，然后修改文件名和文件内的 sensor name。如需增加新的分辨率，可以参考以下代码。

```

1  } constructCustStaticMetadata_PLATFORM_PROJECT_SENSOR_DRVNAME_MAX96712_MIPI_YUVjson  X
src > multimedia > camera-hal > mt8668 > mtkcam-core > external > firmware > metadata > max96712_mipi_yuv > {} constructCustStaticMetadata_PLATFORM_PRO
2
3  "function_name": "constructCustStaticMetadata_PLATFORM_PROJECT_SENSOR_DRVNAME_MAX96712_MIPI_YUV",
4  "metadata": [
5  {
6    "condition": "always",
7    "type": "CONFIG_METADATA",
8    "tag": "MTK_REQUEST_AVAILABLE_CAPABILITIES",
9    "values": [
10     {"value": "MTK_REQUEST_AVAILABLE_CAPABILITIES_BACKWARD_COMPATIBLE", "type": "MUINT8"}
11   ],
12   },
13   {
14     "condition": "always",
15     "type": "CONFIG_METADATA",
16     "tag": "MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_WITH_DURATIONS",
17     "values": [
18       {"value": 33, "type": "MINT64"},
19       {"value": 2288, "type": "MINT64"},
20       {"value": 1280, "type": "MINT64"},
21       {"value": "MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT", "type": "MINT64"},
22       {"value": 33333333, "type": "MINT64"},
23       {"value": 33333333, "type": "MINT64"},
24       {"value": 33, "type": "MINT64"},
25       {"value": 1920, "type": "MINT64"},
26       {"value": 1088, "type": "MINT64"},
27       {"value": "MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT", "type": "MINT64"},
28       {"value": 33333333, "type": "MINT64"},
29       {"value": 33333333, "type": "MINT64"},
30       {"value": 33, "type": "MINT64"},
31       {"value": 1920, "type": "MINT64"},
32       {"value": 1080, "type": "MINT64"},
33       {"value": "MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT", "type": "MINT64"},
34       {"value": 33333333, "type": "MINT64"},
35       {"value": 33333333, "type": "MINT64"}
36     ]
37   }
38 ]
39 }

```

1.3.1.8 配置 Sensor Format

根据 bridge 输出的 format, 配置 UYVY 或者 YUYV:

```

\src\kernel\modules\mt8668\camera\imgsensor\src_spm-isp8s\
common\max96712 mipi yuv\max96712mipiyuv Sensor.c

```

查找所有配置 sensor output dataformat 的地方改为实际的 format。

```
.sensor_output_dataformat = SENSOR_OUTPUT_FORMAT_UYVY,
```

1.3.1.9 配置 Modestruct

与供应商确认设置的相关参数，并将其填入结构体中。

Pclk->pixel clock, 单位 Hz

Linelength->HTS

Framelength->VTS

linelength × framelength × fps= pclk

Grabwindow_width-> camera size 宽 (在 MT8668 上没有这个值)

Grabwindow_height->camera size 高 (在 MT8668 上没有这个值)

Mipi_pixel_rate-> mipi datarate × lane_number/bitdepth, YUV bitdepth = 16

```
static struct subdrv_mode_struct mode_struct[] = {
{
    .frame_desc = frame_desc_prev,
    .num_entries = ARRAY_SIZE(frame_desc_prev),
    .mode_setting_table = PARAM_UNDEFINED,
    .mode_setting_len = PARAM_UNDEFINED,
    .seamless_switch_group = PARAM_UNDEFINED,
    .seamless_switch_mode_setting_table = PARAM_UNDEFINED,
    .seamless_switch_mode_setting_len = PARAM_UNDEFINED,
    .hdr_mode = HDR_NONE,
    .raw_cnt = 1,
    .exp_cnt = 1,
    .pclk = 340032000,
    .linelength = 8096,
    .framelength = 1400,
    .max_framerate = 300,
    .mipi_pixel_rate = 340032000,
    .readout_length = 0,
    .read_margin = 10,
    .framelength_step = 2,
    .coarse_integ_step = 2,
    .sensor_output_dataformat = SENSOR_OUTPUT_FORMAT_UYVY,
    .multi_exposure_shutter_range[IMSENSOR_EXPOSURE_LET].min = 8,
    .imgsensor_winsize_info = {
        .full_w = SUPERFRAME_WIDTH,
        .full_h = SUPERFRAME_HEIGHT,
        .x0_offset = 0,
        .y0_offset = 0,
        .w0_size = SUPERFRAME_WIDTH,
        .h0_size = SUPERFRAME_HEIGHT,
        .scale_w = SUPERFRAME_WIDTH,
        .scale_h = SUPERFRAME_HEIGHT,
        .x1_offset = 0,
        .y1_offset = 0,
        .w1_size = SUPERFRAME_WIDTH,
        .h1_size = SUPERFRAME_HEIGHT,
        .x2_offset = 0,
        .y2_offset = 0,
        .w2_size = SUPERFRAME_WIDTH,
        .h2_size = SUPERFRAME_HEIGHT,
    },
    .pdaf_cap = TRUE,
    .imgsensor_pd_info = NULL,
    .ae_binning_ratio = 1465,
    .fine_integ_line = 826,
    .delay_frame = 3,
    .csi_param = {
        .legacy_phy = 0,
        .not_fixed_trail_settle = 1,
        .not_fixed_dphy_settle = 1,
        .dphy_trail = 4,
        .dphy_data_settle = 36,
        .cphy_settle = 36,
        .clk_lane_no_initial_flow = 0,
    },
    .dpc_enabled = true, /* reg 0x0b06 */
}, /* preview */
```

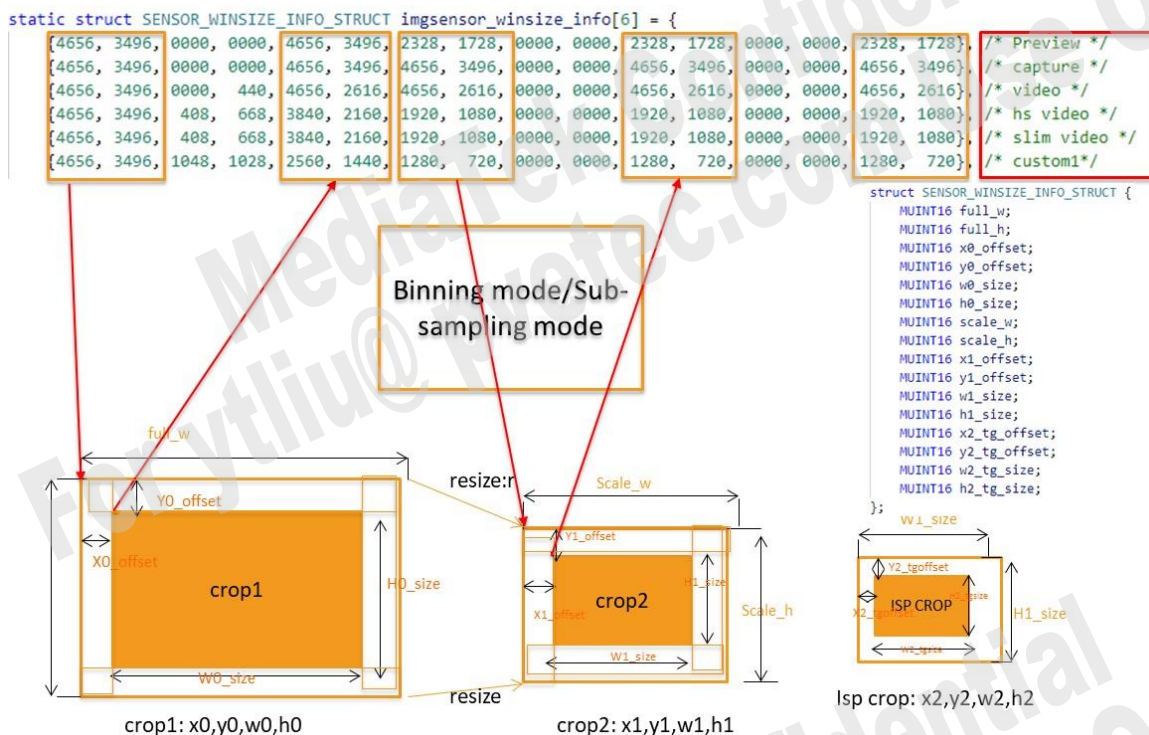
pclk,linelenth,framelength
framerate,mipi pixel rate
在这里配置

format在这里配置

winsize info 在这里配置

hs trail & settle 在这里配置

下图是借用了之前版本的一个对 winsize info 的解释, 来描述各个字段的含义:



限制: full size (4:3) 或 full size (16:9) 的宽和高需要进行 4 倍数对齐。

配置 sensor 输出的宽高，与 Grabwindow_width/Grabwindow_height 的大小一致。

Grab window 要求：

1. 设置的宽必须是 16 的倍数，高必须为 4 的倍数。
2. 相同比例下视角一致：4:3 与 16:9 要求 sensor 输出的 window 保证水平方向和 full size 视角一致。
3. Grab window 设置建议和 sensor 输出的 window 一致。

特殊情况下是：grabwindow_width(height) ≤ sensor output width(height)-startx(y)

1.3.1.10 配置 static_ctx

如下图所示：

1. sensor_id 字段需填对应的 sensor ID。
2. i2c_add_table 一般只配置 0x52 即可，即实际的解串器的地址。
3. 配置 mipi_lane_num，这里表示是 MIPI 的 lane 的个数。
4. cam_type 是按照模组场景的类型填写，如果是单 sensor 的话，可以填 MTK_SENSOR_TYPE_SINGLE，它有如下值可以选择：
 - SENSOR_TYPE_COMB_AVM 表示组帧 AVM

- SENSOR_TYPE_MUTI 表示 multicam
- SENSOR_TYPE_NON_COMB_AVM 表示非组帧 AVM

5. sensor_output_dataformat 表示模组输出的数据的 format, 填实际的 format 即可

```
static struct subdrv_static_ctx static_ctx = {
    .sensor_id = MAX96712_MIPI_SENSOR_ID,
    .reg_addr_sensor_id = {0x0000},
    .i2c_addr_table = {0x52, 0xFF},
    .i2c_burst_write_support = TRUE,
    .i2c_transfer_data_type = I2C_DT_ADDR_16_DATA_8,
    .eeprom_info = 0,
    .eeprom_num = 0,
    // .resolution = {SUPERFRAME_WIDTH, SUPERFRAME_HEIGHT},
    .mirror = IMAGE_HV_MIRROR,

    // .clk = 24,
    // .isp_driving_current = ISP_DRIVING_4MA,
    .sensor_interface_type = SENSOR_INTERFACE_TYPE_MIPI,
    .mipi_sensor_type = MIPI_OPHY_NCSI2,
    .mipi_lane_num = SENSOR_MIPI_4_LANE,
    .ob_pedestal = 0x40,
    .cam_type = SENSOR_TYPE_NON_COMB_AVM,
    .sensor_output_dataformat = SENSOR_OUTPUT_FORMAT_UYVY,
};
```

6. multicam_group: 表示是第几组 group, 0 和 1 是保留给 single 或者 AVM 的驱动来填写的。

所以如果是第一组 multicam, 这个字段要填 2, 如果是第二组 multicam, 这个字段要填 3, 依次类推。

默认这个字段最大只能是 3。如果需要增加, 需要修改代码 adaptor-drv.c 里的数组 multicam_info_table 的 size。

1.3.1.11 配置 VC 信息

如果是单个 VC, 可以进行如下配置:

```
301 static struct mtk_mbus_frame_desc_entry frame_desc_prev[] = {
302     {
303         .bus.csi2 = {
304             .channel = 0,
305             .data_type = 0x1E, //yuv422
306             .hsize = FRAME_WIDTH,
307             .vsize = FRAME_HEIGHT,
308             .fs_seq = MTK_FRAME_DESC_FS_SEQ_FIRST,
309             // .user_data_desc = VC_RAW_DATA,
310         },
311     },
312 };
```

如果是有多组 VC, 可以进行如下配置, 主要是修改 channel 和 user_data_desc:

```

310 static struct mtk_mbus_frame_desc_entry frame_desc_prev[] = {
311     {
312         .bus.csi2 = {
313             .channel = 0,
314             .data_type = 0x1E, //yuv422
315             .hsize = FRAME_WIDTH,
316             .vsize = FRAME_HEIGHT,
317             .fs_seq = MTK_FRAME_DESC_FS_SEQ_FIRST,
318             .user_data_desc = VC_BRIDGE_RAW_0,
319         },
320     },
321     {
322         .bus.csi2 = {
323             .channel = 1,
324             .data_type = 0x1E, //yuv422
325             .hsize = FRAME_WIDTH,
326             .vsize = FRAME_HEIGHT,
327             .fs_seq = MTK_FRAME_DESC_FS_SEQ_FIRST,
328             .user_data_desc = VC_BRIDGE_RAW_1,
329         },
330     },
331     {
332         .bus.csi2 = {
333             .channel = 2,
334             .data_type = 0x1E, //yuv422
335             .hsize = FRAME_WIDTH,
336             .vsize = FRAME_HEIGHT,
337             .fs_seq = MTK_FRAME_DESC_FS_SEQ_FIRST,
338             .user_data_desc = VC_BRIDGE_RAW_2,
339         },
340     },

```

1.3.1.12 配置获取 Sensor ID 函数

```

00249: static int return_sensor_id(struct subdrv_ctx *ctx)
00250: {
00251:     return read_cmos_sensor(ctx, MAX96712_ID_REG);
00252: }
00253:

```

1.3.1.13 配置 Sensor Init 函数

下面的函数内主要是配置模組的设置，需要客户和供应商调试生成。

```
89 static int max96712_sensor_init(struct subdrv_ctx *ctx)
90 {
91
92 »     DRV_LOG(ctx, "--MAX96712-ISX019-sensor_init_start\n");
93 »     »     write_cmos_sensor(ctx, 0x0017, 0x14);
94 »     »     write_cmos_sensor(ctx, 0x0019, 0x94);
95 »     »     write_cmos_sensor(ctx, 0x0013, 0x75);
```

1.3.1.14 配置 get_sensor_usage 函数

```
668
669 static int get_sensor_usage(struct subdrv_ctx *ctx, enum mtk_sensor_usage *usage)
670 {
671 »     *usage = MTK_SENSOR_USAGE_NONCOMB;
672 »     return 0;
673 }
```

它的值可以选择下面几种，COMB 表示组帧 AVM，MUTI 表示 multicam，NONCOMB 表示非组帧 AVM，单 sensor 的话配置为 SINGLE。

```
67 enum mtk_sensor_usage {
68 »     MTK_SENSOR_USAGE_SINGLE,
69 »     MTK_SENSOR_USAGE_COMB,
70 »     MTK_SENSOR_USAGE_MUTI,
71 »     MTK_SENSOR_USAGE_NONCOMB,
72 };
73
```

1.3.1.14.1 Streaming Control

初始化的设定全部放到 sensor_init() 函数里。MIPI enable 和 disable 函数放到 streaming_control() 函数中。

```
static int set_streaming_control(void *arg, bool enable)
{
    struct subdrv_ctx *ctx = (struct subdrv_ctx *)arg;

    DRV_LOG(ctx, "streaming_enable(0=Sw Standby,1=streaming): %d\n", enable);
    if (enable) {
        /*turn on mipi*/
        write_cmos_sensor(ctx, 0x08A0, 0x84);
        write_cmos_sensor(ctx, 0x040B, 0x52);
    } else {
        /*turn off mipi*/
        write_cmos_sensor(ctx, 0x08A0, 0x04);
        write_cmos_sensor(ctx, 0x040B, 0x00);
    }
    mdelay(20);
    return ERROR_NONE;
}
```

1.3.1.14.2 subdrv_static_ctx 的成员解析

Member	Meaning
sensor_id	Sensor ID define in kd_imgsensor.h.
reg_addr_sensor_id	Sensor register address where sensor ID is read. Up to 3 bytes.

Member	Meaning
i2c_addr_table	I2C write ID, end in 0xff, 4 for maximum. E.g., i2c_addr_table = {0x20, 0x6e, 0xff},.
eeeprom_info	The address of eeeprom_info_struct. If there is no eeeprom device, set to 0.
eeeprom_num	Array size of the eeeprom_info_struct. If there is no eeeprom device, set to 0.
resolution	Full pixel size of sensor output.
mirror	Set IMAGE_HV_MIRROR if the sensor orientation is 180 degrees difference from module orientation.
mclk	Overwrite MCLK frequency (MHz)
isp_driving_current	Overwrite MCLK driving current (mA)
sensor_interface_type	Data output interface type. Set default MIPI.
mipi_sensor_type	C-PHY or D-PHY.
mipi_lane_num	How many lanes/trios of MIPI-PHY.
ob_pedestal	OB offset. General is 64.
sensor_output_dataformat	Bayer order, 4-cell order and SW/HW remo.
ana_gain_def	Default analog gain. Set 4x. 1024base.
ana_gain_min	Minimum analog gain. 1024base.
ana_gain_max	Maximum analog gain. 1024base.
ana_gain_type	Sony:type 0; OV:type 1; Samsung:type 2; Hynix:type 3; GC:type 4
ana_gain_step	Minimum valid step of analog gain. 1024base.
ana_gain_table	Use valid analog gain table. Remove unsuitable value which is not linearity. 1024base.
ana_gain_table_size	Size of analog gain table.
min_gain_iso	Set minimum ISO 100.
exposure_def	Default exposure line. Set 0x3D0.
exposure_min	Minimum exposure line.
exposure_max	Maximum exposure line.
exposure_step	Minimum valid step of exposure line.
exposure_margin	Maximum margin of exposure line.
frame_length_max	Maximum framelength.
ae_effective_frame	AE effective frame
frame_time_delay_frame	The frame "frame length" setting take effect. Sony sensor filled in 3; other sensors filled in 2.
start_exposure_offset	Parameter tuned in CTS sensor fusion test.
pdaf_type	Reference enum IMGSENSOR_PDAF_SUPPORT_TYPE_ENUM.
hdr_type	Reference enum IMGSENSOR_HDR_SUPPORT_TYPE_ENUM.
seamless_switch_support	If sensor supports seamless switch, set 1 to enable seamless switch function.
temperature_support	Set to 1 if sensor supports temperature sensor readout.
g_temp	Implement get temperature function.

Member	Meaning
g_gain2reg	Implement analog gain to register mapping rule.
s_gph	Implement set enable/disable group hold function.
s_cali	Implement the function to write data from eeprom_info_struct to sensor (e.g., QSC, cross-talk, ...)
reg_addr_stream	Sensor register address where streaming on/off is controlled.
reg_addr_mirror_flip	Sensor register address where mirror/flip on/off is controlled. No use when set to 0.
reg_addr_exposure	Sensor register address where set exposure line. Up to 3 channels of exposure. Up to 3 bytes.
long_exposure_support	Set to 1 if sensor supports long exposure left shift function.
reg_addr_exposure_lshift	Sensor register address where long exposure left shift is set.
reg_addr_ana_gain	Sensor register address where analog gain is set. Up to 3 channels of exposure. Up to 3 bytes.
reg_addr_frame_length	Sensor register address where framelength is set.
reg_addr_temp_en	If "temperature_support" is set to 1. Sensor register address where temperature sensor on/off is controlled.
reg_addr_temp_read	If "temperature_support" is set to 1. Sensor register address where temperature output is read.
reg_addr_auto_extend	For Sony sensor, sensor register address where auto extend function enable/disable is controlled.
reg_addr_frame_count	For Samsung sensor, sensor register address where frame count is read to determine whether sensor stream is off or not.
reg_addr_fast_mode	For Sony sensor, sensor register address where fast mode on/off is controlled.
init_setting_table	The address of sensor initial setting table.
init_setting_len	Array size of the sensor initial setting table.
mode	The address of subdrv_mode_struct.
sensor_mode_num	Array size of the subdrv_mode_struct.
list	The address of customized feature control list.
list_len	Array size of the customized feature control list.
checksum_value	The calculated value when Test Pattern output, for Camera Auto Test.
multicam_group	Multicam group number

1.3.1.15 Sensor Mode 配置

Subdrv_mode_struct mode_struct[] 这个结构体数组是用来存不同 Sensor Mode 的数据配置的。

下面以 Preview 为例子说明了一些需要客户修改的字段，对于 YUV sensor，一般后面的 Sensor Mode 和 Preview 是

完全一样的。对于 Raw sensor 可能会有差异，根据实际需要来配置。

```
static struct subdrv_mode_struct mode_struct[] = {
{
    .frame_desc = frame_desc_prev,
    .num_entries = ARRAY_SIZE(frame_desc_prev),
    .mode_setting_table = PARAM_UNDEFINED,
    .mode_setting_len = PARAM_UNDEFINED,
    .seamless_switch_group = PARAM_UNDEFINED,
    .seamless_switch_mode_setting_table = PARAM_UNDEFINED,
    .seamless_switch_mode_setting_len = PARAM_UNDEFINED,
    .hdr_mode = HDR_NONE,
    .raw_cnt = 1,
    .exp_cnt = 1,
    .pclk = 340032000,
    .linelength = 8096,
    .framelength = 1400,
    .max_framerate = 300,
    .mipi_pixel_rate = 340032000,
    .readout_length = 0,
    .read_margin = 10,
    .framelength_step = 2,
    .coarse_integ_step = 2,
    .sensor_output_dataformat = SENSOR_OUTPUT_FORMAT_UYVY,
    .multi_exposure_shutter_range[IMGSSENSOR_EXPOSURE_LE].min = 8,
    .imgsensor_winsize_info = {
        .full_w = SUPERFRAME_WIDTH,
        .full_h = SUPERFRAME_HEIGHT,
        .x0_offset = 0,
        .y0_offset = 0,
        .w0_size = SUPERFRAME_WIDTH,
        .h0_size = SUPERFRAME_HEIGHT,
        .scale_w = SUPERFRAME_WIDTH,
        .scale_h = SUPERFRAME_HEIGHT,
        .x1_offset = 0,
        .y1_offset = 0,
        .w1_size = SUPERFRAME_WIDTH,
        .h1_size = SUPERFRAME_HEIGHT,
        .x2_tg_offset = 0,
        .y2_tg_offset = 0,
        .w2_tg_size = SUPERFRAME_WIDTH,
        .h2_tg_size = SUPERFRAME_HEIGHT,
    },
    .pdacap = 1800,
    .imgsensor_pd_info = NULL,
    .ae_binning_ratio = 1465,
    .fine_integ_line = 826,
    .delay_frame = 3,
    .csi_param = {
        .legacy_phy = 0,
        .not_fixed_trail_settle = 1,
        .not_fixed_dphy_settle = 1,
        .dphy_trail = 4,
        .dphy_data_settle = 36,
        .cphy_settle = 36,
        .clk_lane_no_initial_flow = 0,
    },
    .dpc_enabled = true, /* reg 0x0b06 */
}, /* preview */
};
```

pclk,linelenth,framelength
framerate,mipi pixel rate
在这里配置

format在这里配置

winsize info 在这里配置

hs trail & settle 在这里配置

1.3.1.16 Driver 功能

1.3.1.16.1 Driver 操作函数 List

Driver 函数的 list 如下：

```
789 static struct subdrv_ops ops = {
790     .get_id = get_imgsensor_id,
791     .init_ctx = init_ctx,
792     .open = open,
793     .get_info = common_get_info,
794     .get_resolution = common_get_resolution,
795     .control = common_control,
796     .feature_control = common_feature_control,
797     .close = common_close,
798     .get_frame_desc = common_get_frame_desc,
799     .get_temp = common_get_temp,
800     .get_csi_param = common_get_csi_param,
801     .vsync_notify = vsync_notify,
802     .update_sof_cnt = common_update_sof_cnt,
803     .parse_ebd_line = common_parse_ebd_line,
804     .get_sensor_usage = get_sensor_usage,
805 };
806
```

在以 adapter 名字开头的 imgsensor 的代码里，其往往是通过 subdrv_call 的方式来调用到 driver 代码里的，就是上面的函数 list，比如：

```
ret = subdrv_call(ctx, get_id, &sensor_id);
```

上面显示的调用就会 call 到上面函数 list 里的 get_sensor_id。

上面 list 里有很多 common 开头的函数，这个部分是用的通用的处理流程。不需要在 sensor driver 里特别实现，下面就不做过多介绍了。

针对贵司可能改到的 get_imgsensor_id 和 open 函数，请参考下面两个章节。

1.3.1.16.2 get_imgsensor_id 函数

开机 search sensor 时会通过 get_imgsensor_id 函数读取 ID。若能成功读到 ID，在 UI 上显示 Camera APP 的图标。

下面函数主要有两个功能：一获取 sensor ID，二判断是否有连接到模组。

```

675 static int get_imgsensor_id(struct subdrv_ctx *ctx, u32 *sensor_id)
676 {
677     u8 i = 0;
678     u8 retry;
679
680     ctx->i2c_write_id = static_ctx.i2c_addr_table[0];
681     retry = 2;
682
683     while (static_ctx.i2c_addr_table[i] != 0xff) {
684         spin_lock(&imgsensors_drv_lock);
685         ctx->i2c_write_id = static_ctx.i2c_addr_table[i];
686         spin_unlock(&imgsensors_drv_lock);
687         do {
688             *sensor_id = return_sensor_id(ctx);
689             if (*sensor_id == (static_ctx.sensor_id)) {
690                 DRV_LOG(ctx, "i2c write id: 0x%x, sensor id: 0x%x\n",
691                     ctx->i2c_write_id, *sensor_id);
692                 return ERROR_NONE;
693             }
694             DRV_LOG(ctx, "Read id fail, id: 0x%x, i2c id: 0x%x\n",
695                 *sensor_id, ctx->i2c_write_id);
696             retry--;
697         } while (retry > 0);
698         i++;
699         retry = 2;
700     }
701     if (*sensor_id != (static_ctx.sensor_id)) {
702         *sensor_id = 0xFFFFFFFF;
703         return ERROR_SENSOR_CONNECT_FAIL;
704     }
705     return ERROR_NONE;
706 }
707 }

```

1.3.1.16.3 Open 函数

每次进 camera 时会调用。

读 sensor_id，确认 I2C 通信是否正常。

调用 sensor_init 函数初始化 ctx 结构体中的一些变量。

```

688 static int open(struct subdrv_ctx *ctx)
689 {
690     u8 i = 0;
691     u8 retry = 2;
692     u16 sensor_id = 0;
693     u32 scenario_id = 0;
694
695     while (static_ctx.i2c_addr_table[i] != 0xff) {
696         spin_lock(&imgsensor_drv_lock);
697         ctx->i2c_write_id = static_ctx.i2c_addr_table[i];
698         spin_unlock(&imgsensor_drv_lock);
699         do {
700             sensor_id = return_sensor_id(ctx);
701             if (sensor_id == MAX96712ISX_SENSOR_ID) {
702                 DRV_LOG(ctx, "i2c write id: 0x%x, sensor id: 0x%x\n",
703                     ctx->i2c_write_id, sensor_id);
704                 break;
705             }
706             DRV_LOG(ctx, "Read id fail, id: 0x%x, i2c id: 0x%x\n",
707                 sensor_id, ctx->i2c_write_id);
708             retry--;
709         } while (retry > 0);
710         i++;
711         if (sensor_id == MAX96712ISX_SENSOR_ID)
712             break;
713         retry = 2;
714     }
715     if (sensor_id != MAX96712ISX_SENSOR_ID)
716         return -ERROR_SENSOR_CONNECT_FAIL;
717
718     /* initail sequence write in */
719     max96712_sensor_init(ctx);
720     retry = 3;
721     do {
722         if (Chenck_Sensor_Signal(ctx)) {
723             DRV_LOG(ctx, "Sensor Not Connect");
724             mdelay(5);
725             retry--;
726         } else
727             break;
728     } while (retry > 0);
729
730     if (retry == 0) {
731         DRV_LOG(ctx, "OPEN FAIL!Sensor Not Connect");
732         return -ERROR_SENSOR_CONNECT_FAIL;
733     }
734
735     /* move camera init to initctx(), void max9286 init too much time */
736
737     spin_lock(&imgsensor_drv_lock);
738     memset(ctx->exposure, 0, sizeof(ctx->exposure));
739     memset(ctx->ana_gain, 0, sizeof(ctx->gain));
740     ctx->exposure[0] = ctx->s_ctx.exposure_def;
741     ctx->ana_gain[0] = ctx->s_ctx.ana_gain_def;
742     ctx->current_scenario_id = scenario_id;
743     ctx->pclk = mode_struct[scenario_id].pclk;
744     ctx->line_length = mode_struct[scenario_id].linelength;
745     ctx->frame_length = mode_struct[scenario_id].framelength;
746     ctx->frame_length_rg = ctx->frame_length;
747     ctx->current_fps = ctx->pclk / ctx->line_length * 10 / ctx->frame_length;
748     ctx->readout_length = mode_struct[scenario_id].readout_length;
749     ctx->read_margin = mode_struct[scenario_id].read_margin;
750     ctx->min_frame_length = ctx->frame_length;
751     ctx->autoflicker_en = FALSE;
752     ctx->test_pattern = 0;
753     ctx->ihdr_mode = 0;
754     ctx->pdaf_mode = 0;
755     ctx->hdr_mode = 0;
756     ctx->extend_frame_length_en = 0;
757     ctx->is_seamless = 0;
758     ctx->fast_mode_on = 0;
759     ctx->sof_cnt = 0;
760     ctx->ref_sof_cnt = 0;
761     ctx->is_streaming = 0;
762     spin_unlock(&imgsensor_drv_lock);
763
764     return ERROR_NONE;
765 }
766
767 }

```

注:

1. 关于分辨率的说明

由于场景不同，给到 User space 的图的大小和实际 seninf 收取的 VC 的大小不同，所以在 Driver 里会有两个分辨率，客户可以以 max96712isx 这个非组帧 AVM 的 Driver 为例。

搜索下面的宏进行对照配置。

```
#define VC 0

#define SUPERFRAME_WIDTH 5120
#define SUPERFRAME_HEIGHT 720

#define FRAME_WIDTH 1280
#define FRAME_HEIGHT 720

#define MAX96712_ID_REG 0x00U
#define SC22AT_I2C_ADDR 0x60
#define MAX96717_I2C_ADDR 0x80
#define MAX96712_SENSOR_ID 0x52

#define LOG_INF(format, args...) pr_info(PFX "[%s] " format, __func__, ##args)
#define LOG_DBG(format, args...) pr_debug(PFX "[%s] " format, __func__, ##args)
#define LOG_ERR(format, args...) pr_info(PFX "[%s] " format, __func__, ##args)
#define read_cmos_sensor(...) subdrv_i2c_rd_u8(__VA_ARGS__)
#define write_cmos_sensor(...) subdrv_i2c_wr_u8(__VA_ARGS__)

#define read_cmos_sensor_with_i2cid(subctx, i2c_id, reg) \
```

这是上层看到的最大分辨率，如非组帧AVM，这里的width会是5120

这是单个VC的分辨率

2. 关于 dts 里 GPIO 供电的部分，还可以选择 regulator 封装的方式。

用法如下：

```
gpio163_regulator: gpio-163 {
    compatible = "regulator-fixed";
    regulator-name = "gpio163_regulator";
    regulator-min-microvolt = <1800000>;
    regulator-max-microvolt = <1800000>;
    gpio = <gpio 163 0x0>;
    enable-active-high;
    status = "okay";
};

sensor2: sensor2@52 {
    compatible = "mediatek,imgsensord";
    sensor-names = "max96712_mipi_yuv";
    reg = <0x52>;

    #thermal-sensor-cells = <0>;
    dvdd-supply = <gpio165_regulator>;
    pdn-supply = <gpio163_regulator>;
    avdd1-supply = <i2c20_regulator>;
    avdd2-supply = <i2c21_regulator>;
    avdd3-supply = <i2c22_regulator>;
    avdd4-supply = <i2c23_regulator>;

    clocks = <cksys_reg_clk CLK_CKSYS_REG_UNIVPLL_192M_D32>,
        <cksys_reg_clk CLK_CKSYS_REG_UNIVPLL_192M_D16>,
        <cksys_reg_clk CLK_CKSYS_REG_F26M_CLK_D2>,
        <cksys_reg_clk CLK_CKSYS_REG_UNIVPLL_192M_D10>,
        <cksys_reg_clk CLK_CKSYS_REG_UNIVPLL_192M_D8>,
        <cksys_reg_clk CLK_CKSYS_REG_TCK_26M_MX9>,
        <cksys_reg_clk CLK_CKSYS_REG_OSC_D10>,
        <cksys_reg_clk CLK_CKSYS_REG_UNIVPLL_D6_D8>,
        <cksys_reg_clk CLK_CKSYS_REG_CAMTGT1_SEL>;
    clock-names = "6", "12", "13", "19.2", "24", "26", "26_ulposc", "52", "mclk";

    status = "okay";

    port {
        sensor2_out: endpoint {
            remote-endpoint = <seninf_csi_port_4_in>;
        };
    };
};
```

3. UT 测试指令

```
C:\Users>adb shell sentest_v4l2 //just search sensor, not streaming
```

```
[show_Sensors]sensorNum 2
[show_Sensors]name:SENSOR_DRVNAME_OV05A20_MIPI_RAW type:0
[show_Sensors]index:0, SensorDevIdx:1
[show_Sensors]name:SENSOR_DRVNAME_IMX576_MIPI_RAW type:0
[show_Sensors]index:1, SensorDevIdx:2
[main]Param: 1 <sensorDev> <scenario> <fps>
[main]<sensorDev> : main(1), Sub(2), Main2(4), sub2(8), Main3(16)
[main]<scenario> : Pre(0), Cap(1), VD(2), slim1(3), slim2(4)
```

显示 sensorNum 不为 0，且展示了传感器的具体名称，说明传感器搜索成功。接下来测试传感器的输出。

```
C:\Users> adb shell sentest_v4l2 1 0 //search sensor, then open main (1) sensor preview (0)
```

```
[test_SensorInterface]name:SENSOR_DRVNAME_MAX96712_MIPI_YUV type:1
[test_SensorInterface]index:1, SensorDevIdx:4
[test_SensorInterface]scenario(0) , binning type = 1465
[test_SensorInterface]scenario(0) , rawFmtType = 0
[test_SensorInterface]scenario(0) , OB_pedestal = 0
[test_SensorInterface]scenario(0) , sensorFormatOrder = 4
[test_SensorInterface]scenario(0) , seamless_targets_count = 0
[test_SensorInterface]Sensor index:1, scenarioId:0
[test_SensorInterface]crop w:5120, h:720
[test_SensorInterface]HDR_Support:0
[test_SensorInterface]scenario(0) , RGBW_Support = 0
[test_SensorInterface]scenario(0) , PDAF_Support = 0, dpc(1), pdc(0)
[test_SensorInterface]scenario(0) , Def_MirrorFlip = 3
[test_SensorInterface]scenario(0) , sensorFormatCellType = 0
[test_SensorInterface]scenario(0) , isAwbEnabled = 0
[test_SensorInterface]SENSOR_CMD_GET_SENSOR_CROP_WIN_INFO ret 0
[test_SensorInterface]SENSOR_CMD_GET_SENSOR_RGBW_OUTPUT_MODE rgbw mode = 0
[test_SensorInterface]ana gain min/max = 1024/65536
[test_SensorInterface]dig gain min/max = 1024/16384
[test_SensorInterface]dig gain step = 4
[test_SensorInterface]SENSOR_CMD_GET_OFFSET_TO_START_OF_EXPOSURE exposure_offset = 3000000
[test_SensorInterface]pHalSensor->powerOn start
[test_CID][test g_volatile_ctrl] scenario (by set format)
[test_CID][V4L2_CID_VBLANK] ret:0 VB:680
[test_CID][V4L2_CID_HBLANK] ret:0 HB:2976
[test_CID][V4L2_CID_MTK_SENSOR_PIXEL_RATE] ret:0 pixel rate:500000000
[test_CID][V4L2_CID_MTK_STAGGER_INFO] ret:0 count:0 order:
[test_CID][V4L2_CID_MAX_EXP_TIME]ret 0 exposure 0, max_exp_time 0
[test_CID][V4L2_CID_MAX_EXP_TIME]ret 0 exposure 1, max_exp_time 0
[test_CID][V4L2_CID_MAX_EXP_TIME]ret 0 exposure 2, max_exp_time 0
[streamon]sensor: idx=2 devpath=/dev/v4l-subdev41 drvname=max96712_mipi_yuv
[streamon]seninf: devpath=/dev/v4l-subdev0 cid=0x98f901 en=1
[test_SensorInterface]main/rolling_shutter=11658240

[test_SensorInterface]main/exp_margin=777
[test_SensorInterface]main/atv-Continuous=1
[test_SensorInterface]main/atv-BypassSensorScenario=0
[test_SensorInterface]main/min_shutter=24 shutter_step=2
```

查看 seninf 状态的指令如下，查到结果后，可以发给联发科技确认。

```
root@auto8668p164:/# cat /sys/devices/platform/soc/1a400000.seninf-top/status

[seninf-2] port 3 intf 3 test 0 cphy 0 lanes 4
source sensor1 flags 0x4

[seninf-3] port 2 intf 2 test 0 cphy 0 lanes 0

[seninf-9] port 0 intf 0 test 0 cphy 0 lanes 0
source sensor11 flags 0x4

[seninf-8] port 0 intf 0 test 0 cphy 0 lanes 0
source sensor10 flags 0x4

[seninf-7] port 0 intf 0 test 0 cphy 0 lanes 0
source sensor9 flags 0x4

[seninf-6] port 0 intf 0 test 0 cphy 0 lanes 0
source sensor8 flags 0x4

[seninf-5] port 0 intf 0 test 0 cphy 0 lanes 0
source sensor7 flags 0x4

[seninf-4] port 1 intf 1 test 0 cphy 0 lanes 4
source sensor2 flags 0x5
csirx_mac_csi2 irq_stat 0x00000324, irq_g1_stat 0x00000000, seninf_async_irq 0x00000000
csirx_mac_csi2 CTRL/_EXP/_RCV/_ERR:[0] (0x002b0011)/(0x00000000)/(0x00000000)/(0x00000000)
csirx_mac_csi2 CTRL/_EXP/_RCV/_ERR:[1] (0x002b0111)/(0x00000000)/(0x00000000)/(0x00000000)
csirx_mac_csi2 CTRL/_EXP/_RCV/_ERR:[2] (0x002b0211)/(0x00000000)/(0x00000000)/(0x00000000)
csirx_mac_csi2 CTRL/_EXP/_RCV/_ERR:[3] (0x002b0311)/(0x00000000)/(0x00000000)/(0x00000000)
csirx_mac_csi2 CTRL/_EXP/_RCV/_ERR:[4] (0x00300011)/(0x00000000)/(0x00000000)/(0x00000000)
csirx_mac_csi2 CTRL/_EXP/_RCV/_ERR:[5] (0x00300111)/(0x00000000)/(0x00000000)/(0x00000000)
csi2_line_frame_num 0x00000000
csi2_packet_status 0x0a00031e
csi2_packet_cnt_status 0x007c0a33
rx-ana settle ck 0x09 dt 0x24
rx-ana trail en 1 param 0x04
CDPHY_RX_ANA_SETTING_0 0x00000022 ANA_SETTING_1 0x40630244 DPHY_RX_SPARE0 0x00000031
CSIRX_MAC_CSI2_RESYNC_MERGE_CTRL 0x0000011f MERGE_CTRL2 0x00308000
MipiRx_ANA1:CDPHY_RX_ANA_AD_0/_1:(0x33)/(0x0),AD_HS_0/_1/_2:(0x0)/(0x008)/(0x0)
MipiRx_ANA1:CDPHY_RX_ANA_9:(0x413f0000)/ANA_8:(0x203fff00)/ANA_7:(0x4090909)/ANA_6:(0x4090909)/ANA_5:(0x48)
data_not_enough_cnt : <0>
err_lane_resync_cnt : <0>
crc_err_cnt : <0>
ecc_err_double_cnt : <0>
ecc_err_corrected_cnt : <0>
[0] vc 0x0 dt 0x1e outmux 1 tag: 0
    exp size 2560x720
    rec size 2561x720
    done_irq 0x1
    ref_vsync_irq 0x1
    TAG_DBG_PORT 0xa0002d0 TAG_CRC_PORT 0xffff, OUTMUX_SW_CFG_DONE 0x1
[1] vc 0x1 dt 0x1e outmux 2 tag: 0
    exp size 2560x720
    rec size 2561x720
    done_irq 0x1
    ref_vsync_irq 0x1
    TAG_DBG_PORT 0xa0002d0 TAG_CRC_PORT 0xffff, OUTMUX_SW_CFG_DONE 0x1
[2] vc 0x2 dt 0x1e outmux 3 tag: 0
    exp size 2560x720
    rec size 2561x720
    done_irq 0x1
    ref_vsync_irq 0x1
    TAG_DBG_PORT 0xa0002d0 TAG_CRC_PORT 0xffff, OUTMUX_SW_CFG_DONE 0x1
[3] vc 0x3 dt 0x1e outmux 4 tag: 0
    exp size 2560x720
    rec size 2561x720
    done_irq 0x1
    ref_vsync_irq 0x1
    TAG_DBG_PORT 0xa0002d0 TAG_CRC_PORT 0xffff, OUTMUX_SW_CFG_DONE 0x1
root@auto8668p164:/#
```

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。